

The Gracefulness of Graph $B(m, n)$

GUO WENFU, Translation, [comments](#), and code by Pascal Welke 

Abstract. In this paper, we shall prove that the graphs $B(m, n) = C_m \cup P_n$ are **graceful** when $m \equiv 1$ or $2 \pmod{4}$, where $C_m = A_1A_2 \dots A_mA_1$ and $P_n = A_1B_1B_2 \dots B_n$ ($m \geq 3, n > 0$).

Keywords: cycle, path, tadpole, graceful graph

I have obtained the original Chinese version [1] of this article with the help of colleagues and translated it with the help of Google Gemini as of December 2025. I have checked the proof sketches in the translation carefully and have implemented all constructions in Python. All functions are named after the respective parts of the original paper and construct a sequence of vertices that are injectively labeled with numbers from 1 to $m + n$ and a sequence of edges. As the vertex labeling is injective, I used the labels as node ids. The code is provided next to the descriptions of the original paper and is available on [github](#).

1 INTRODUCTION

Let the **cycle** be $C_m = A_1A_2 \dots A_mA_1$ and the **path** be $P_n = A_1B_1B_2 \dots B_n$. We define the graph $B(m, n) = C_m \cup P_n$ ($m \geq 3, n > 0$).

It has been proven in [2] that $B(m, n)$ is a 1-graceful graph. It was also proven that $B(m, n)$ is a graceful graph when $m \equiv 0$ or $3 \pmod{4}$. This paper gives an affirmative answer to the question: “Is $B(m, n)$ a graceful graph when $m \equiv 1$ or $2 \pmod{4}$ and $n > 0$?”.

DEFINITION 1 (GRACEFUL LABELING). For a simple graph $G(V, E)$ we assign non-negative integers $\varphi(v) \in \{1, 2, \dots, |E|\}$ to every vertex $v \in V$ which we call vertex labels. We call the value $L(u, v) = |\varphi(u) - \varphi(v)|$ the edge label of the edge uv . A simple graph G is called a graceful graph if it is labeled such that:

- (1) Different vertices have different vertex labels.
- (2) Different edges have different edge labels.

The set of edge labels for a graceful graph is $\{1, 2, \dots, |E|\}$.

The entrypoint to the python code is the following function that constructs a graceful labeling for any tadpole with $m \equiv 1$ or $2 \pmod{4}$.

```
1 def graceful_tadpole(m, n):
2     '''
3     Implements all the constructions presented in the paper
4     Guo Wenfu. The Gracefulness of Graph B(m,n).
5     Journal of Inner Mongolia Normal University (Natural Science Edition),
6     1994(2): 25-29.
7     '''
8     assert(m >= 3)
9     assert(m % 4 == 2 or m % 4 == 1)
10
11     if m % 4 == 2:
12         return thm1(m, n)
13     if m % 4 == 1:
14         return thm2(m, n)
```

2 CASE $m \equiv 2 \pmod{4}$

THEOREM 1. For all $m \equiv 2 \pmod{4}$ and $n > 0$, $B(m, n)$ is a *graceful graph*.

PROOF. Let $m = 4l + 2$, which implies $l = \frac{m-2}{4}$. A graceful labeling for $n = 1$ and arbitrary $l > 0$ is given in Figure 1 [with comments and python code in Section 2.1](#). Similarly, a graceful labeling for $n = 2$ and arbitrary $l > 0$ is given in Figure 2 [with comments and python code in Section 2.2](#). For the case $n \geq 3$ we will construct graceful labelings in Section 2.3. $\square$

```

1 def thm1(m, n):
2     assert(m >= 3)
3     assert(m % 4 == 2)
4     assert(n > 0)
5
6     if n == 1:
7         return fig1(m, n)
8     if n == 2:
9         return fig2(m, n)
10    else:
11        if n % 6 == 3:
12            return case_3_1(m, n, fig1)
13        if n % 6 == 4:
14            return case_3_2(m, n, fig1)
15        if n % 6 == 5:
16            return case_3_3(m, n, fig1)
17        if n % 6 == 0:
18            return case_3_4(m, n, fig1)
19        if n % 6 == 1:
20            return case_3_5(m, n, fig1)
21        if n % 6 == 2:
22            return case_3_6(m, n, fig1)

```

2.1 Case 1: $n = 1$

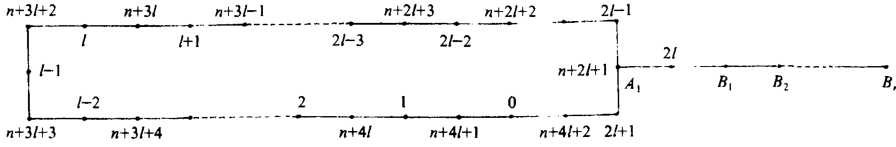


Fig. 1. Labeling for $B(m, 1)$ when $m \equiv 2 \pmod{4}$

```

1 def fig1(m, n):
2     assert(m % 4 == 2)
3     # careful. n >= 3 requires additional tail nodes
4     # added in another step
5     assert(n == 1 or n >= 3)
6
7     l = m // 4
8
9     vertices = list()

```

```

10
11 # point A1 will be at index 0, followed by m-1 cycle nodes, clockwise.
12 vertices.append(n + 2*l + 1)
13 vertices.append(2*l + 1)
14
15 for i in range(l+1):
16     vertices.append(n + 4*l + 2 - i)
17     vertices.append(i)
18
19 for i in range(l+1, 2*l+1):
20     vertices.append(n + 4*l + 1 - i) # after l, we shift this by one.
21     vertices.append(i)
22
23 edges = list(zip(vertices, vertices[1:] + [vertices[0]]))
24
25 # Then 1 path node
26 vertices.append(2*l)
27
28 # and the edge connecting it to the cycle
29 edges.append((vertices[0], vertices[m])) # edge connecting path and cycle
30
31 return vertices, edges

```

2.2 Case 2: $n = 2$

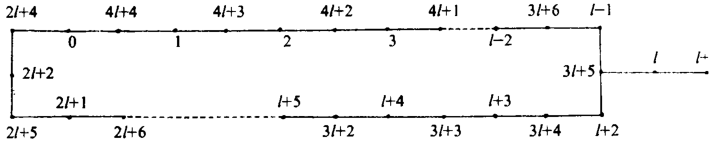


Fig. 2. Labeling for $B(m, 2)$ when $m \equiv 2 \pmod{4}$

```

1 def fig2(m, n):
2     assert(m % 4 == 2)
3     assert(n == 2)
4
5     l = m // 4
6     vertices = list()
7
8     # point A1 will be at index 0, followed by m-1 cycle nodes, counterclockwise.
9     vertices.append(3*l+5)
10
11     # we create increasing edge weights [2l+6, ..., m+n] by alternatingly
12     # increasing and decreasing the vertex label.
13     # the last node constructed by this loop is the vertex with label 0
14     startlength_counterclockwise = 2*l + 6 # 3*l + 5 - (l - 1)
15     decrease = True
16     for w in range(startlength_counterclockwise, m+n+1):
17         current = vertices[-1]
18         if decrease:

```

```

19         vertices.append(current - w)
20         decrease = False
21     else:
22         vertices.append(current + w)
23         decrease = True
24
25     vertices.append(2*l + 4)
26
27     # we create increasing edge weights [2, ..., 2l+2] by alternately
28     # increasing and decreasing the vertex label.
29     # the last node constructed by this loop is the vertex with label l+2
30     decrease = True
31     for w in range(2, 2*l+3):
32         current = vertices[-1]
33         if decrease:
34             vertices.append(current - w)
35             decrease = False
36         else:
37             vertices.append(current + w)
38             decrease = True
39
40     # create cycle edges
41     edges = list(zip(vertices, vertices[1:] + [vertices[0]]))
42
43     # Then 2 path nodes
44     vertices.append(l)
45     vertices.append(l+1)
46
47     # create tail edges
48     edges.append((vertices[0], vertices[m])) # edge connecting path and cycle
49     edges.append((vertices[m], vertices[m+1])) # path edge
50
51     return vertices, edges

```

2.3 Case 3: $n \geq 3$

For the case $n \geq 3$ we will construct graceful labelings as follows:

- First, $C_m \cup \{B_1\}$ is labeled according to Figure 1.
- The vertex labels for $C_m \cup \{B_1\}$ are distinct and their set is $\{0, 1, \dots, 2l+1\} \cup \{n+2l+1, n+2l+2, \dots, n+3l\} \cup \{n+3l+2, \dots, n+4l+1, n+4l+2\}$.
- The edge labels are distinct and their set is $\{n, n+1, \dots, n+4l+1, n+4l+2\}$.
- The remaining path is $B_1B_2 \dots B_n$, and the available edge label set is $\{1, 2, \dots, n-1\}$.
- The labeling for the path $B_1B_2 \dots B_n$ is then given across six cases based on $n \pmod{6}$.

Labeling for Path $B_1B_2 \dots B_n$ (for $n \geq 3$)

Case (3-1) Let $n = 6s + 3$, $a = \varphi(A_1) = n + 2l + 1$, and $b = \varphi(B_1) = 2l$. The general assignments for $0 \leq i \leq s - 1$ are:

$$\begin{aligned}
 \varphi(B_{6i+1}) &= b + 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6i+2}) &= a - 2 - 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6i+3}) &= b + 2 + 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6i+4}) &= a - 1 - 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6i+5}) &= b + 4 + 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6i+6}) &= a - 3 - 3i & 0 \leq i \leq s - 1 \\
 \varphi(B_{6s+1}) &= 2l + 3s \\
 \varphi(B_{6s+2}) &= 2l + 2 + 3s \\
 \varphi(B_{6s+3}) &= 2l + 3 + 3s
 \end{aligned} \tag{1}$$

It is easy to see that the labels are all distinct, and they consist of all integers from $2l$ to $2l + n$ with $2l + 1$ removed, and the label of B_1 is still $2l$. Comparing with the set of labels on $C_m \cup A_1 \cup B_1$, we can conclude that the labels on $C_m \cup P_m$ are also all distinct, and do not exceed $|E| = n + m = n + 4l + 2$.

Calculating the number of edge labels, we have:

$$\begin{aligned}
 L(B_{6i+1}B_{6i+2}) &= n - 1 - 6i \\
 L(B_{6i+2}B_{6i+3}) &= n - 3 - 6i \\
 L(B_{6i+3}B_{6i+4}) &= n - 2 - 6i \\
 L(B_{6i+4}B_{6i+5}) &= n - 4 - 6i \\
 L(B_{6i+5}B_{6i+6}) &= n - 6 - 6i \\
 L(B_{6i+6}B_{6i+7}) &= n - 5 - 6i
 \end{aligned} \tag{2}$$

Here $0 \leq i \leq s - 1$. In the above six formulas, letting $i = 0, 1, \dots, s - 1$ respectively, we see that the edge labels are all distinct and form the $n - 3$ edge labels from 3 to $n - 1$.

Moreover, since $L(B_{6s+1}B_{6s+2}) = 2$ and $L(B_{6s+2}B_{6s+3}) = 1$, it follows that the set of edge labels of $B_1, B_2, \dots, B_n$ is $\{1, 2, \dots, n - 1\}$.

Considering the case on $C_m \cup P_n$ as a whole, it is easy to see that the labeling method we give is graceful.

The proof for the other cases below is similar; we only present the key steps and omit the rest.

```

1  def case_3_1(m, n, fig):
2      assert(n % 6 == 3)
3
4      head_vertices, head_edges = fig(m, n)
5
6      tail_vertices = list()
7      l = m // 4
8      s = (n - 3) // 6
9      a = n + 2 * l + 1
10     b = 2 * l
11     for i in range(s):
12         tail_vertices.append(b + 3 * i)
13         tail_vertices.append(a - 2 - 3 * i)
14         tail_vertices.append(b + 2 + 3 * i)

```

```

15         tail_vertices.append(a-1-3*i)
16         tail_vertices.append(b+4+3*i)
17         tail_vertices.append(a-3-3*i)
18     tail_vertices.append(b+3*s)
19     tail_vertices.append(2*l+2+3*s)
20     tail_vertices.append(2*l+3+3*s)
21
22     # construct tail edges
23     tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
24
25     # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,
26     # as it is already part of the output of fig1
27     return head_vertices + tail_vertices[1:], head_edges + tail_edges

```

Case (3-2) When $n = 6s + 4$, the labels for B_1 to B_{6s+1} still use formula (1). Additionally, we set:

$$\varphi(B_{6s+2}) = 2l + 3 + 3s$$

$$\varphi(B_{6s+3}) = 2l + 2 + 3s$$

$$\varphi(B_{6s+4}) = 2l + 4 + 3s$$

The edge labels calculated by formula (2) still apply, but the set of edge labels is now $\{4, 5, \dots, n-1\}$. Furthermore, because

$$L(B_{6s+1}B_{6s+2}) = 3, \quad L(B_{6s+2}B_{6s+3}) = 1, \quad L(B_{6s+3}B_{6s+4}) = 2,$$

Therefore, the set of edge labels for the path $B_1B_2 \dots B_n$ is $\{1, 2, \dots, n-1\}$.

```

1  def case_3_2(m, n, fig):
2      assert(n % 6 == 4)
3
4      head_vertices, head_edges = fig(m, n)
5
6      tail_vertices = list()
7      l = m // 4
8      s = (n - 3) // 6
9      a = n+2*l+1
10     b = 2*l
11     for i in range(s):
12         tail_vertices.append(b+3*i)
13         tail_vertices.append(a-2-3*i)
14         tail_vertices.append(b+2+3*i)
15         tail_vertices.append(a-1-3*i)
16         tail_vertices.append(b+4+3*i)
17         tail_vertices.append(a-3-3*i)
18     tail_vertices.append(b+3*s)
19     tail_vertices.append(2*l+3+3*s)
20     tail_vertices.append(2*l+2+3*s)
21     tail_vertices.append(2*l+4+3*s)
22
23     # construct tail edges
24     tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
25
26     # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,

```

```

27     # as it is already part of the output of fig1
28     return head_vertices + tail_vertices[1:], head_edges + tail_edges

```

Case (3-3) When $n = 6s + 5$, the labels for B_1 to B_{6s+1} still use formula (1). Additionally, we set:

$$\varphi(B_{6s+2}) = 2l + 4 + 3s$$

$$\varphi(B_{6s+3}) = 2l + 3 + 3s$$

$$\varphi(B_{6s+4}) = 2l + 5 + 3s$$

$$\varphi(B_{6s+5}) = 2l + 2 + 3s$$

The edge labels calculated by formula (2) still apply, but the set of edge labels is now $\{5, 6, \dots, n - 1\}$. Furthermore, due to

$$L(B_{6s+1}B_{6s+2}) = 4$$

$$L(B_{6s+2}B_{6s+3}) = 1$$

$$L(B_{6s+3}B_{6s+4}) = 2$$

$$L(B_{6s+4}B_{6s+5}) = 3$$

the set of edge labels for the path $B_1B_2 \dots B_n$ is $\{1, 2, \dots, n - 1\}$.

```

1  def case_3_3(m, n, fig):
2      assert(n % 6 == 5)
3
4      head_vertices, head_edges = fig(m, n)
5
6      tail_vertices = list()
7      l = m // 4
8      s = (n - 3) // 6
9      a = n+2*l+1
10     b = 2*l
11     for i in range(s):
12         tail_vertices.append(b+3*i)
13         tail_vertices.append(a-2-3*i)
14         tail_vertices.append(b+2+3*i)
15         tail_vertices.append(a-1-3*i)
16         tail_vertices.append(b+4+3*i)
17         tail_vertices.append(a-3-3*i)
18     tail_vertices.append(b+3*s)
19     tail_vertices.append(2*l+4+3*s)
20     tail_vertices.append(2*l+3+3*s)
21     tail_vertices.append(2*l+5+3*s)
22     tail_vertices.append(2*l+2+3*s)
23
24     # construct tail edges
25     tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
26
27     # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,
28     # as it is already part of the output of fig1
29     return head_vertices + tail_vertices[1:], head_edges + tail_edges

```

Case (3-4) When $n = 6s + 6$, the labels for B_1 to B_{6s+1} still use formula (1). Additionally, set:

$$\varphi(B_{6s+2}) = 2l + 5 + 3s$$

$$\varphi(B_{6s+3}) = 2l + 2 + 3s$$

$$\varphi(B_{6s+4}) = 2l + 6 + 3s$$

$$\varphi(B_{6s+5}) = 2l + 4 + 3s$$

$$\varphi(B_{6s+6}) = 2l + 3 + 3s$$

The edge labels calculated by formula (2) still apply, but the set of edge labels is now $\{6, 7, \dots, n - 1\}$. Furthermore, because

$$L(B_{6s+1}B_{6s+2}) = 5$$

$$L(B_{6s+2}B_{6s+3}) = 3$$

$$L(B_{6s+3}B_{6s+4}) = 4$$

$$L(B_{6s+4}B_{6s+5}) = 2$$

$$L(B_{6s+5}B_{6s+6}) = 1$$

Therefore, the set of edge labels for the path $B_1B_2 \dots B_n$ is $\{1, 2, \dots, n - 1\}$.

```

1 def case_3_4(m, n, fig):
2     assert(n % 6 == 0)
3
4     head_vertices, head_edges = fig(m, n)
5
6     tail_vertices = list()
7     l = m // 4
8     s = (n - 3) // 6
9     a = n+2*l+1
10    b = 2*l
11    for i in range(s):
12        tail_vertices.append(b+3*i)
13        tail_vertices.append(a-2-3*i)
14        tail_vertices.append(b+2+3*i)
15        tail_vertices.append(a-1-3*i)
16        tail_vertices.append(b+4+3*i)
17        tail_vertices.append(a-3-3*i)
18    tail_vertices.append(b+3*s)
19    tail_vertices.append(2*l+5+3*s)
20    tail_vertices.append(2*l+2+3*s)
21    tail_vertices.append(2*l+6+3*s)
22    tail_vertices.append(2*l+4+3*s)
23    tail_vertices.append(2*l+3+3*s)
24
25    # construct tail edges
26    tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
27
28    # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,
29    # as it is already part of the output of fig1
30    return head_vertices + tail_vertices[1:], head_edges + tail_edges

```


Case (3-5) When $n = 6s + 7$, the labels for B_1 to B_{6s+1} still use formula (1). Additionally, set:

$$\varphi(B_{6s+2}) = 2l + 6 + 3s$$

$$\varphi(B_{6s+3}) = 2l + 2 + 3s$$

$$\varphi(B_{6s+4}) = 2l + 7 + 3s$$

$$\varphi(B_{6s+5}) = 2l + 4 + 3s$$

$$\varphi(B_{6s+6}) = 2l + 5 + 3s$$

$$\varphi(B_{6s+7}) = 2l + 3 + 3s$$

The edge labels calculated by formula (2) still apply, but the set of edge labels is now $\{7, 8, \dots, n - 1\}$. Furthermore, because

$$L(B_{6s+1}B_{6s+2}) = 6$$

$$L(B_{6s+2}B_{6s+3}) = 4$$

$$L(B_{6s+3}B_{6s+4}) = 5$$

$$L(B_{6s+4}B_{6s+5}) = 3$$

$$L(B_{6s+5}B_{6s+6}) = 1$$

$$L(B_{6s+6}B_{6s+7}) = 2$$

the set of edge labels for the path $B_1B_2 \dots B_n$ is $\{1, 2, \dots, n - 1\}$.

```

1 def case_3_5(m, n, fig):
2     assert(n % 6 == 1)
3
4     head_vertices, head_edges = fig(m, n)
5
6     tail_vertices = list()
7     l = m // 4
8     s = (n - 3) // 6
9     a = n+2*l+1
10    b = 2*l
11    for i in range(s):
12        tail_vertices.append(b+3*i)
13        tail_vertices.append(a-2-3*i)
14        tail_vertices.append(b+2+3*i)
15        tail_vertices.append(a-1-3*i)
16        tail_vertices.append(b+4+3*i)
17        tail_vertices.append(a-3-3*i)
18    tail_vertices.append(b+3*s)
19    tail_vertices.append(2*l+6+3*s)
20    tail_vertices.append(2*l+2+3*s)
21    tail_vertices.append(2*l+7+3*s)
22    tail_vertices.append(2*l+4+3*s)
23    tail_vertices.append(2*l+5+3*s)
24    tail_vertices.append(2*l+3+3*s)
25
26    # construct tail edges
27    tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
28
29    # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,
```

```

30     # as it is already part of the output of fig1
31     return head_vertices + tail_vertices[1:], head_edges + tail_edges

```

Case (3-6) When $n = 6s + 8$, the labels for B_1 to B_{6s+1} still use formula (1). Additionally, set:

$$\varphi(B_{6s+2}) = 2l + 7 + 3s$$

$$\varphi(B_{6s+3}) = 2l + 2 + 3s$$

$$\varphi(B_{6s+4}) = 2l + 8 + 3s$$

$$\varphi(B_{6s+5}) = 2l + 4 + 3s$$

$$\varphi(B_{6s+6}) = 2l + 5 + 3s$$

$$\varphi(B_{6s+7}) = 2l + 3 + 3s$$

$$\varphi(B_{6s+8}) = 2l + 6 + 3s$$

The edge labels calculated by formula (2) still apply, but the set of edge labels is now $\{8, 9, \dots, n-1\}$. Furthermore, because

$$L(B_{6s+1}B_{6s+2}) = 7$$

$$L(B_{6s+2}B_{6s+3}) = 5$$

$$L(B_{6s+3}B_{6s+4}) = 6$$

$$L(B_{6s+4}B_{6s+5}) = 4$$

$$L(B_{6s+5}B_{6s+6}) = 1$$

$$L(B_{6s+6}B_{6s+7}) = 2$$

$$L(B_{6s+7}B_{6s+8}) = 3$$

Therefore, the set of edge labels for the path $B_1B_2 \cdots B_n$ is $\{1, 2, \dots, n-1\}$.

```

1  def case_3_1(m, n, fig):
2      assert(n % 6 == 3)
3
4      head_vertices, head_edges = fig(m, n)
5
6      tail_vertices = list()
7      l = m // 4
8      s = (n - 3) // 6
9      a = n+2*l+1
10     b = 2*l
11     for i in range(s):
12         tail_vertices.append(b+3*i)
13         tail_vertices.append(a-2-3*i)
14         tail_vertices.append(b+2+3*i)
15         tail_vertices.append(a-1-3*i)
16         tail_vertices.append(b+4+3*i)
17         tail_vertices.append(a-3-3*i)
18     tail_vertices.append(b+3*s)
19     tail_vertices.append(2*l+2+3*s)
20     tail_vertices.append(2*l+3+3*s)
21
22     # construct tail edges
23     tail_edges = [(u,v) for u,v in zip(tail_vertices, tail_vertices[1:])]
24

```

```

25     # we are constructing edge labels for B_1, ... B_n, hence we drop B_1,
26     # as it is already part of the output of fig1
27     return head_vertices + tail_vertices[1:], head_edges + tail_edges

```

In all cases, the set of edge labels for the path $B_1B_2 \dots B_n$ is $\{1, 2, \dots, n-1\}$. By combining this with the labels of $C_m \cup A_1B_1$, the entire graph $C_m \cup P_n$ has a graceful labeling.

3 CASE $m \equiv 1 \pmod{4}$

THEOREM 2. For all $m \equiv 1 \pmod{4}$ and $n > 0$, $B(m, n)$ is a *graceful graph*.

PROOF. Let $m = 4l + 1$. If n is odd, we proceed as described in Section 3.1. If n is even, we distinguish three cases in Section 3.2. $\square$

```

1  def thm2(m, n):
2      assert(m >= 3)
3      assert(m % 4 == 1)
4      assert(n > 0)
5
6      k = n // 2
7      l = m // 4
8
9      if n % 2 == 1:
10         return fig3(m, n)
11     if n % 2 == 0:
12         if k == 1:
13             return fig4(m, n)
14         elif k >= 1:
15             return fig5(m, n)
16         elif k < 1:
17             if n % 6 == 4:
18                 return case_3_2(m, n, fig6)
19             if n % 6 == 0:
20                 return case_3_4(m, n, fig6)
21             if n % 6 == 2:
22                 return case_3_6(m, n, fig6)
23     else:
24         raise ValueError(f'Uncaught case: k={k} l={l}, n={n} m={m}')

```

3.1 Case n is odd

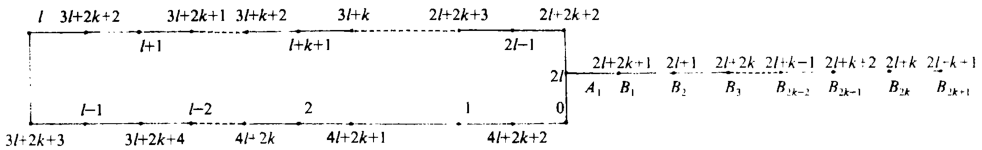


Fig. 3. Labeling for $B(m, n)$ when $m \equiv 1 \pmod{4}$ and $n = 2k + 1$ (odd, $0 \leq k \leq l - 2$)

Let $n = 2k + 1$, where k is a non-negative integer.

- When $0 \leq k \leq l - 2$, a graceful labeling is given in Figure 3.

- When $k > l - 2$, the point labeled $l + k + 1$ in Figure 3 will be on the path $A_1B_1B_2 \dots B_{2k+1}$.

The construction for this case labels the path $C_0C_2 \dots C_{m+n-1}$ with the node labels

$$\varphi(C_i) = \begin{cases} \frac{i}{2} & \text{if } i \text{ is even} \\ n + m - \frac{i-1}{2} & \text{if } i \text{ is odd} \end{cases}$$

for $i < 2l + 2k$ and

$$\varphi(C_i) = \begin{cases} \frac{i}{2} - 1 & \text{if } i \text{ is even} \\ n + m - \frac{i-1}{2} - 1 & \text{if } i \text{ is odd} \end{cases}$$

for $i \geq 2l + 2k$, skipping the edge label $2l$. This yields all edge labels $\{m, \dots, 1\}$ in decreasing order, except $2l$. We note that $\varphi(C_{m-1}) = 2l$ and can hence connect C_0 and C_{m-1} to obtain a graceful labeling for the tadpole $B(m, n)$.

Both variants are implemented in the `fig3` function.

```

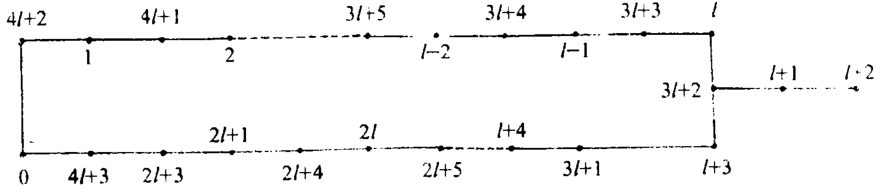
1 def fig3(m, n):
2     assert(m % 4 == 1)
3     assert(n % 2 == 1)
4
5     l = m // 4
6
7     vertices = [0]
8     decrease = False
9     offset = 0
10    for w in range(n+m, 1, -1):
11        current = vertices[-1]
12        if w == 2*l:
13            offset = -1
14        if decrease:
15            vertices.append(current - (w + offset))
16            decrease = False
17        else:
18            vertices.append(current + w + offset)
19            decrease = True
20
21    # construct a long path
22    edges = [(u,v) for u,v in zip(vertices, vertices[1:])]
23    # add the edge connecting 0 with 2l that closes the head cycle
24    edges.append((vertices[0], vertices[m-1]))
25
26    return vertices, edges

```

3.2 Case n is even

Let $n = 2k$, where k is a natural number.

Case (2-1) If $k = 1$ ($n = 2$), a graceful labeling is given in Figure 4.

Fig. 4. Labeling for $B(m, 2)$ when $m \equiv 1 \pmod{4}$ and $n = 2$

```

1 def fig4(m, n):
2     assert(m % 4 == 1)
3     assert(n == 2)
4
5     l = m // 4
6
7     vertices = [l+2, l+1]
8
9     decrease = False
10    for w in range(2*l+1, 4*l+4, 1):
11        current = vertices[-1]
12        if decrease:
13            vertices.append(current - w)
14            decrease = False
15        else:
16            vertices.append(current + w)
17            decrease = True
18
19    if 2*l+3 != 3*l+2:
20        vertices.append(2*l+3)
21
22    decrease = True
23    for w in range(2, 2*l-1, 1):
24        current = vertices[-1]
25        if decrease:
26            vertices.append(current - w)
27            decrease = False
28        else:
29            vertices.append(current + w)
30            decrease = True
31
32    # construct a long path
33    edges = [(u,v) for u,v in zip(vertices, vertices[1:])]
34    # add the edge connecting 3l+2 with l+3 that closes the head cycle
35    edges.append((vertices[-1], vertices[2]))
36
37    return vertices, edges

```

Case (2-2) If $k \geq l$ ($k \neq 1$), graceful labeling is given in Figure 5.

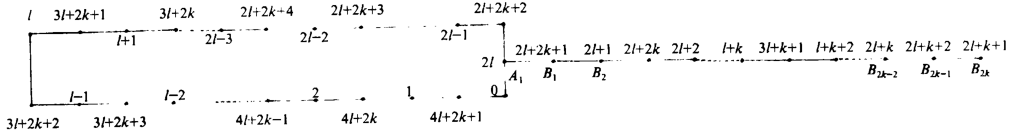


Fig. 5. Labeling for $B(m, n)$ when $m \equiv 1 \pmod{4}$ and $n = 2k$ (even, $k \geq 1$)

```

1 def fig5(m, n):
2     assert(m % 4 == 1)
3     assert(n % 2 == 0)
4
5     k = n // 2
6     l = m // 4
7
8     assert(k >= 1)
9     assert(k != 1)
10
11     vertices = [2*l + k + 1]
12     decrease = False
13     offset = 0
14     for w in range(1, 4*l+2*k+1, 1):
15         current = vertices[-1]
16         # skip edge length 2l
17         if w == 2*l:
18             offset = 1
19         if decrease:
20             vertices.append(current - (w + offset))
21             decrease = False
22         else:
23             vertices.append(current + w + offset)
24             decrease = True
25
26     # construct a long path
27     edges = [(u,v) for u,v in zip(vertices, vertices[1:])]
28     # add the edge connecting 0 with 2l that closes the head cycle
29     edges.append((vertices[-1], vertices[n]))
30
31     return vertices, edges

```

Case (2-3) If $1 < k < l$:

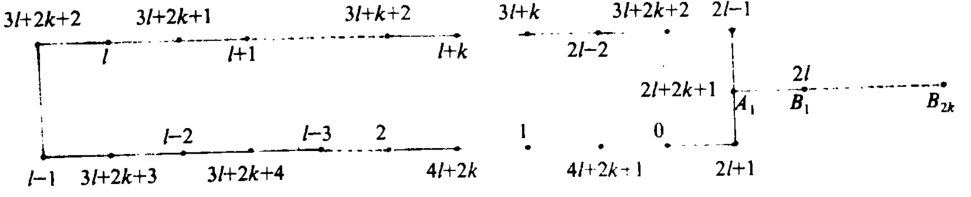


Fig. 6. Labeling for $C_m \cup A_1 B_1$ when $m \equiv 1 \pmod{4}$ and $n = 2k$ (even, $1 < k < l$)

- First, $C_m \cup A_1 B_1$ is labeled according to Figure 6.
- The vertex label set for $C_m \cup A_1 B_1$ is $\{0, 1, \dots, 2l+1\} \cup \{2l+2k+1, \dots, 3l+k\} \cup \{3l+k+2, \dots, 4l+2k+1\}$.
- The edge label set is $\{2k, 2k+1, \dots, 4l+2k+1\}$.
- The remaining path is $B_1 B_2 \dots B_n$, and its required edge label set is $\{1, 2, \dots, 2k-1\}$.
- Since n is even, the construction methods from Theorem 1 for the even n cases—(3-2), (3-4), and (3-6)—can be used to give a graceful labeling for the remaining path.

```

1 def fig6(m, n):
2     assert(m % 4 == 1)
3     assert(n % 2 == 0)
4
5     k = n // 2
6     l = m // 4
7
8     assert (k < l)
9     assert (k > 1)
10
11     vertices = [2*l+1, 0]
12     decrease = False
13     offset = 0
14     for w in range(4*l+2*k+1, 2*k+2, -1):
15         current = vertices[-1]
16         # skip edge length 2*l+1
17         if w == 2*l+1:
18             offset = -1
19         if decrease:
20             vertices.append(current - (w + offset))
21             decrease = False
22         else:
23             vertices.append(current + w + offset)
24             decrease = True
25
26     # add cycle edges
27     edges = list(zip(vertices, vertices[1:] + [vertices[0]]))
28
29     # add tail
30     vertices.append(2*l)
31     edges.append((vertices[-1], vertices[-2]))

```

```

32 |
33 |         return vertices, edges

```

4 REFERENCES

- (1) Guo Wenfu. The Gracefulness of Graph $B(m, n)$. Journal of Inner Mongolia Normal University (Natural Science Edition), 1994(2): 25-29.
- (2) Chen Zhizeng. On the $B(m, n, p)$ and $B(m, n)$ 3-optimality. Journal of Inner Mongolia Normal University (Natural Science Edition), 1991(3): 11-19.

A HELPER FUNCTIONS

To check if a constructed graph labeling is indeed graceful, i have written a test function. The python file contains a loop to check if the constructed labelings for tadpoles with $m \in \{3, \dots, 1000\}$ and $n \in \{1, \dots, 1000\}$ are indeed graceful. They are. :)

```

1  def is_graceful(vertices, edges, verbose=False):
2      '''
3      Check if a graph, given by a list of vertex labels
4      and a list of edges is graceful.
5      '''
6
7      # vertex labels in range 0...m (inclusive)?
8      proper_range = min(vertices) >= 0 and max(vertices) <= len(edges)
9
10     # vertex labels injective?
11     injective_vertex_labels = len(set(vertices)) == len(vertices)
12
13     # edge labels injective?
14     labels = set()
15     for e in edges:
16         labels.add(abs(e[0] - e[1]))
17     injective_edge_labels = len(labels) == len(edges)
18
19     if verbose:
20         print(f'Proper vertex range? {proper_range}')
21         print(f'Injective vertex labels? {injective_vertex_labels}')
22         print(f'Injective edge labels? {injective_edge_labels}')
23     return proper_range and injective_vertex_labels and injective_edge_labels

```

B ORIGINAL PAPER

关于图 $B(m, n)$ 的优美性

郭 文 富

(乌兰察布师专数学系)

摘 要 证明了: 当 $m \equiv 1$ 或 $2 \pmod{4}$ 时, $B(m, n) = C_m \bigcup P_n$ 是优美图, 其中 $C_m = A_1 A_2 \cdots A_m A_1$, $P_n = A_1 B_1 B_2 \cdots B_n$ ($m \geq 3, n > 0$).

关键词 圈, 路, 优美图

设圈 $C_m = A_1 A_2 \cdots A_m A_1$, 路 $P_n = A_1 B_1 B_2 \cdots B_n$. 定义 $B(m, n) = C_m \bigcup P_n$ ($m \geq 3, n > 0$). [1] 中已证明 $B(m, n)$ 是 1-优美图. 还证明了, 当 $m \equiv 0$ 或 $3 \pmod{4}$ 时, $B(m, n)$ 是优美图. 然后, 提出“当 $m \equiv 1$ 或 $2 \pmod{4}$, $n > 0$ 时, $B(m, n)$ 是否优美图”. 本文对此给出肯定的回答.

对于简单图 $G(V, E)$, $\forall v \in V$, 赋予一个非负整数 $\varphi(v)$, 则称图 G 是标定的, $\varphi(v)$ 称为顶点 v 的标号. $|\varphi(u) - \varphi(v)|$ 称为 uv 的标数, 记为 $L(uv)$, 又设整数集 $I = \{0, 1, 2, \dots, |E|\}$.

定义 若简单图 G 是标定的, 顶点的标号取自 I , 不同顶点的标号不同, 不同边的标数不同, 则称图 G 是优美图.

根据上述定义, 实际上, 边标数集合为 $\{1, 2, \dots, |E|\}$. 我们用 $N_1(N_2)$ 表示全体奇(偶)数的集合. 我们仍采用 [1] 中的术语.

定理1 当 $m \equiv 2 \pmod{4}$, $n > 0$ 时, $B(m, n)$ 是优美图.

证 令 $m \equiv 4l + 2$, 则 $l = \frac{m-2}{4}$.

1. $n = 1$ 时, 可按图1给出优美标号.

2. $n = 2$ 时, 可按图2给出优美标号.

3. $n \geq 3$ 时, 首先, 在 $C_m \bigcup A_1 B_1$ 上按图1标号, 易知标号互不相同, 其数集为 $\{0, 1, \dots, 2l+1\} \cup \{n+2l+1, n+2l+2, \dots, n+3l\} \cup \{n+3l+2, \dots, n+4l+1, n+4l+2\}$, 且边标数互不相同, 其数集为 $\{n, n+1, n+2, \dots, n+4l+1, n+4l+2\}$. 还剩余路 $B_1 B_2 \cdots B_n$, 其边标数集应为 $\{1, 2, \dots, n-1\}$.

其次, 在路 $B_1 B_2 \cdots B_n$ 上分6种情形给出标号.

收稿日期: 1992-04-17, 修改稿于 1993-04-20 收到

(3-1) $n = 6s + 3$ 时, 记 $a = \varphi(A_1) = n + 2l + 1$, $b = \varphi(B_1) = 2l$, 置

$$\left. \begin{aligned} \varphi(B_{6i+1}) &= b + 3i, & 0 \leq i \leq s \\ \varphi(B_{6i+2}) &= a - 2 - 3i, \\ \varphi(B_{6i+3}) &= b + 2 + 3i, \\ \varphi(B_{6i+4}) &= a - 1 - 3i, \\ \varphi(B_{6i+5}) &= b + 4 + 3i, \\ \varphi(B_{6i+6}) &= a - 3 - 3i, \end{aligned} \right\} 0 \leq i \leq s-1 \quad (1)$$

$$\begin{aligned} \varphi(B_{6s+2}) &= 2l + 2 + 3s, \\ \varphi(B_{6s+3}) &= 2l + 3 + 3s. \end{aligned}$$

易知标号互不相同, 而且是由 $2l$ 至 $2l + n$ 中间除去 $2l + 1$ 的所有整数, 且 B_1 的标号仍为 $2l$. 与 $C_m \cup A_1 B_1$ 上的标号集比较, 可知 $C_m \cup P_n$ 上的标号互不相同, 且不超过 $|E| = n + m = n + 4l + 2$.

计算边标数, 则

$$\begin{aligned} L(B_{6i+1} B_{6i+2}) &= n - 1 - 6i, \\ L(B_{6i+2} B_{6i+3}) &= n - 3 - 6i, \\ L(B_{6i+3} B_{6i+4}) &= n - 2 - 6i, \\ L(B_{6i+4} B_{6i+5}) &= n - 4 - 6i, \\ L(B_{6i+5} B_{6i+6}) &= n - 6 - 6i, \\ L(B_{6i+6} B_{6(i+1)+1}) &= n - 5 - 6i, \end{aligned} \quad (2)$$

其中 $0 \leq i \leq s-1$. 以上 6 式中分别令 $i = 0, 1, \dots, s-1$, 可知边标数互不相同, 且组成由 3 到 $n-1$ 的 $n-3$ 个边标数. 又因 $L(B_{6s+1} B_{6s+2}) = 2$, $L(B_{6s+2} B_{6s+3}) = 1$, 于是, 路 $B_1 B_2 \dots B_n$ 的边标数集合为 $\{1, 2, \dots, n-1\}$.

综合 $C_m \cup P_n$ 上的情形, 易知我们给出的标号法是优美的.

以下证法类似, 我们只写出关键步骤, 其余省略.

(3-2) $n = 6s + 4$ 时, 对 B_1 至 B_{6s+1} 仍用(1)式标号, 此外置

$$\varphi(B_{6s+2}) = 2l + 3 + 3s, \quad \varphi(B_{6s+3}) = 2l + 2 + 3s, \quad \varphi(B_{6s+4}) = 2l + 4 + 3s.$$

(2)式计算的边标数仍适用, 但此时边标数集合为 $\{4, 5, \dots, n-1\}$. 又因

$$L(B_{6s+1} B_{6s+2}) = 3, \quad L(B_{6s+2} B_{6s+3}) = 1, \quad L(B_{6s+3} B_{6s+4}) = 2,$$

所以路 $B_1 B_2 \dots B_n$ 的边标数集合为 $\{1, 2, \dots, n-1\}$.

(3-3) $n = 6s + 5$ 时, 对 B_1 至 B_{6s+1} 仍用(1)式标号, 此外置

$$\varphi(B_{6s+2}) = 2l + 4 + 3s, \quad \varphi(B_{6s+3}) = 2l + 3 + 3s, \quad \varphi(B_{6s+4}) = 2l + 5 + 3s,$$

$$\varphi(B_{6s+5}) = 2l + 2 + 3s.$$

(2)式计算的边标数仍适用, 但此时边标数集合为 $\{5, 6, \dots, n-1\}$. 又因

$L(B_{6s+1}B_{6s+2})=4, L(B_{6s+2}B_{6s+3})=1, L(B_{6s+3}B_{6s+4})=2, L(B_{6s+4}B_{6s+5})=3,$
 所以路 $B_1B_2\cdots B_n$ 的边标数集合为 $\{1, 2, \cdots, n-1\}$.

(3-4) $n=6s+6$ 时, 对 B_1 至 B_{6s+1} 仍用(1)式标号, 此外置

$$\varphi(B_{6s+2})=2l+5+3s, \varphi(B_{6s+3})=2l+2+3s, \varphi(B_{6s+4})=2l+6+3s,$$

$$\varphi(B_{6s+5})=2l+4+3s, \varphi(B_{6s+6})=2l+3+3s.$$

(2)式计算的边标数仍适用, 但此时边标数集合为 $\{6, 7, \cdots, n-1\}$. 又因

$$L(B_{6s+1}B_{6s+2})=5, L(B_{6s+2}B_{6s+3})=3, L(B_{6s+3}B_{6s+4})=4,$$

$$L(B_{6s+4}B_{6s+5})=2, L(B_{6s+5}B_{6s+6})=1,$$

所以路 $B_1B_2\cdots B_n$ 的边标数集合为 $\{1, 2, \cdots, n-1\}$.

(3-5) $n=6s+7$ 时, 对 B_1 至 B_{6s+1} 仍用(1)式标号, 此外置

$$\varphi(B_{6s+2})=2l+6+3s, \varphi(B_{6s+3})=2l+2+3s, \varphi(B_{6s+4})=2l+7+3s, \varphi(B_{6s+5})$$

$$=2l+4+3s, \varphi(B_{6s+6})=2l+5+3s, \varphi(B_{6s+7})=2l+3+3s.$$

(2)式计算的边标数仍适用, 但此时边标数集合为 $\{7, 8, \cdots, n-1\}$. 又因

$$L(B_{6s+1}B_{6s+2})=6, L(B_{6s+2}B_{6s+3})=4, L(B_{6s+3}B_{6s+4})=5,$$

$$L(B_{6s+4}B_{6s+5})=3, L(B_{6s+5}B_{6s+6})=1, L(B_{6s+6}B_{6s+7})=2,$$

所以路 $B_1B_2\cdots B_n$ 的边标数集合为 $\{1, 2, \cdots, n-1\}$.

(3-6) $n=6s+8$ 时, 对 B_1 至 B_{6s+1} 仍用(1)式标号, 此外置

$$\varphi(B_{6s+2})=2l+7+3s, \varphi(B_{6s+3})=2l+2+3s, \varphi(B_{6s+4})=2l+8+3s,$$

$$\varphi(B_{6s+5})=2l+4+3s, \varphi(B_{6s+6})=2l+5+3s, \varphi(B_{6s+7})=2l+3+3s,$$

$$\varphi(B_{6s+8})=2l+6+3s.$$

(2)式计算的边标数仍适用, 但此时边标数集合为 $\{8, 9, \cdots, n-1\}$. 又因

$$L(B_{6s+1}B_{6s+2})=7, L(B_{6s+2}B_{6s+3})=5, L(B_{6s+3}B_{6s+4})=6, L(B_{6s+4}B_{6s+5})$$

$$=4, L(B_{6s+5}B_{6s+6})=1, L(B_{6s+6}B_{6s+7})=2, L(B_{6s+7}B_{6s+8})=3,$$

所以路 $B_1B_2\cdots B_n$ 的边标数集合为 $\{1, 2, \cdots, n-1\}$.

证毕

定理2 当 $m \equiv 1(\text{mod } 4), n > 0$ 时, $B(m, n)$ 是优美图.

证 令 $m \equiv 4l+1$.

1. $n \in N_1$. 令 $n=2k+1$, k 为非负整数. 当 $0 \leq k \leq l-2$ 时, 可按图 3 给出优美标号. 当 $k > l-2$ 时, 图 3 中标号为 $l+k+1$ 的点将在路 $A_1B_1B_2\cdots B_{2k+1}$ 上.

2. $n \in N_2$. 令 $n=2k$, k 为自然数.

(2-1) $k=1$ 时, $n=2$, 可按图 4 给出优美标号.

(2-2) $k \geq l(k \neq 1)$ 时, 可按图 5 给出优美标号.

(2-3) $1 < k < l$ 时, 首先在 $C_m \cup A_1B_1$ 上按图 6 标号. 易知标号互不相同, 其数集

为 $\{0, 1, \dots, 2l+1\} \cup \{2l+2k+1, \dots, 3l+k\} \cup \{3l+k+2, \dots, 4l+2k+1\}$, 且边标数集为 $\{2k, 2k+1, \dots, 4l+2k+1\}$, 还剩余路 $B_1 B_2 \dots B_n$, 其边标数集应为 $\{1, 2, \dots, 2k+1\}$.

其次在路 $B_1 B_2 \dots B_n$ 上, 由于 n 为偶数, 故只用定理 1 的证明中 (3-2)、(3-4)、(3-6) 的作法就可给出优美标号.

证毕

参 考 文 献

[1] 陈志增. 关于 $B(m, n, p)$ 和 $B(m, n)$ 的 λ -优美性. 内蒙古师大学报(自然科学汉文版), 1991, 3: 11 ~ 19

The Gracefulness on Graph $B(m, n)$

Guo Wenfu

(Dept. of Math, Wulanchab teachers' college)

Abstract In this paper, we shall prove that "the graphs $B(m, n) = C_m \bigcup P_n$ are graceful when $m \equiv 1$ or $2 \pmod{4}$, in which $C_m = A_1 A_2 \dots A_m A_1$, $P_n = A_1 B_1 B_2 \dots B_n$ ($m \geq 3, n > 0$)

Key words cycle, path, graceful graph

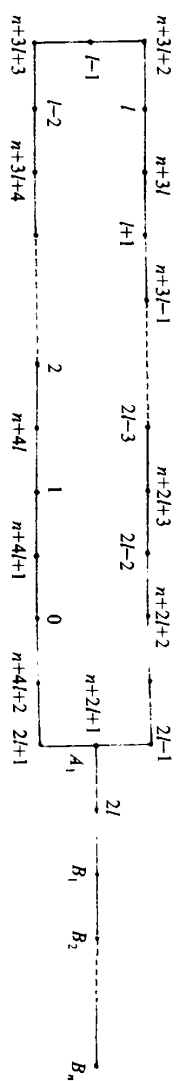


图 1

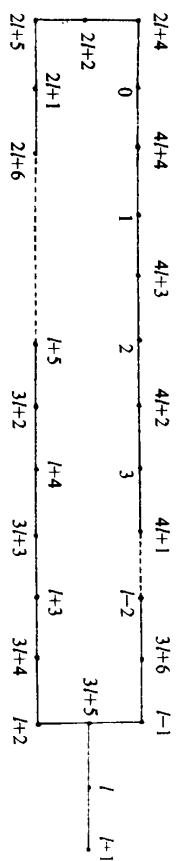


图 2

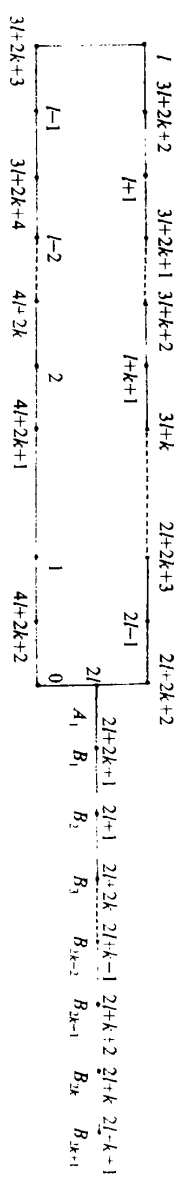


图 3

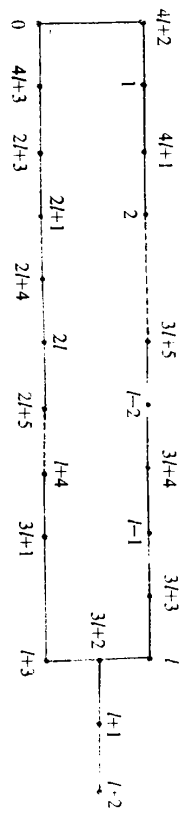


图 4

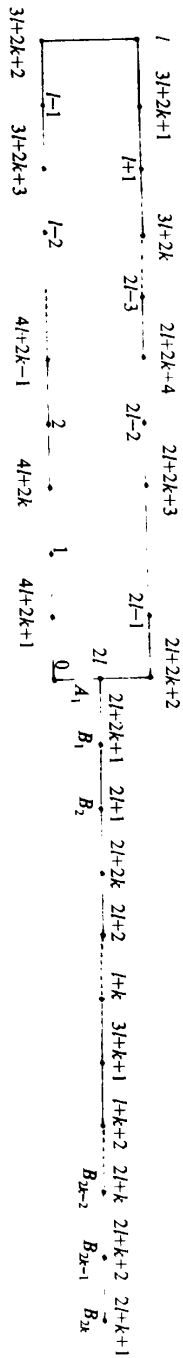


图 5

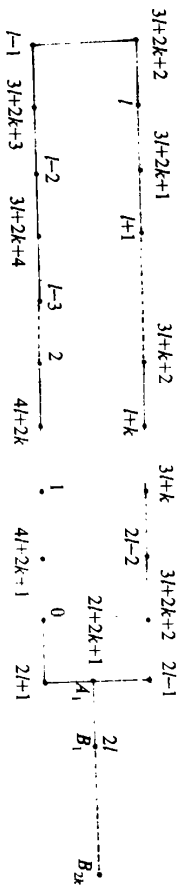


图 6